

FreeRTOS

PRODUCER + CONSUMER 실습

Prelude

FreeRTOS PROSUMER 실습

□ 프로젝트 디렉토리 생성

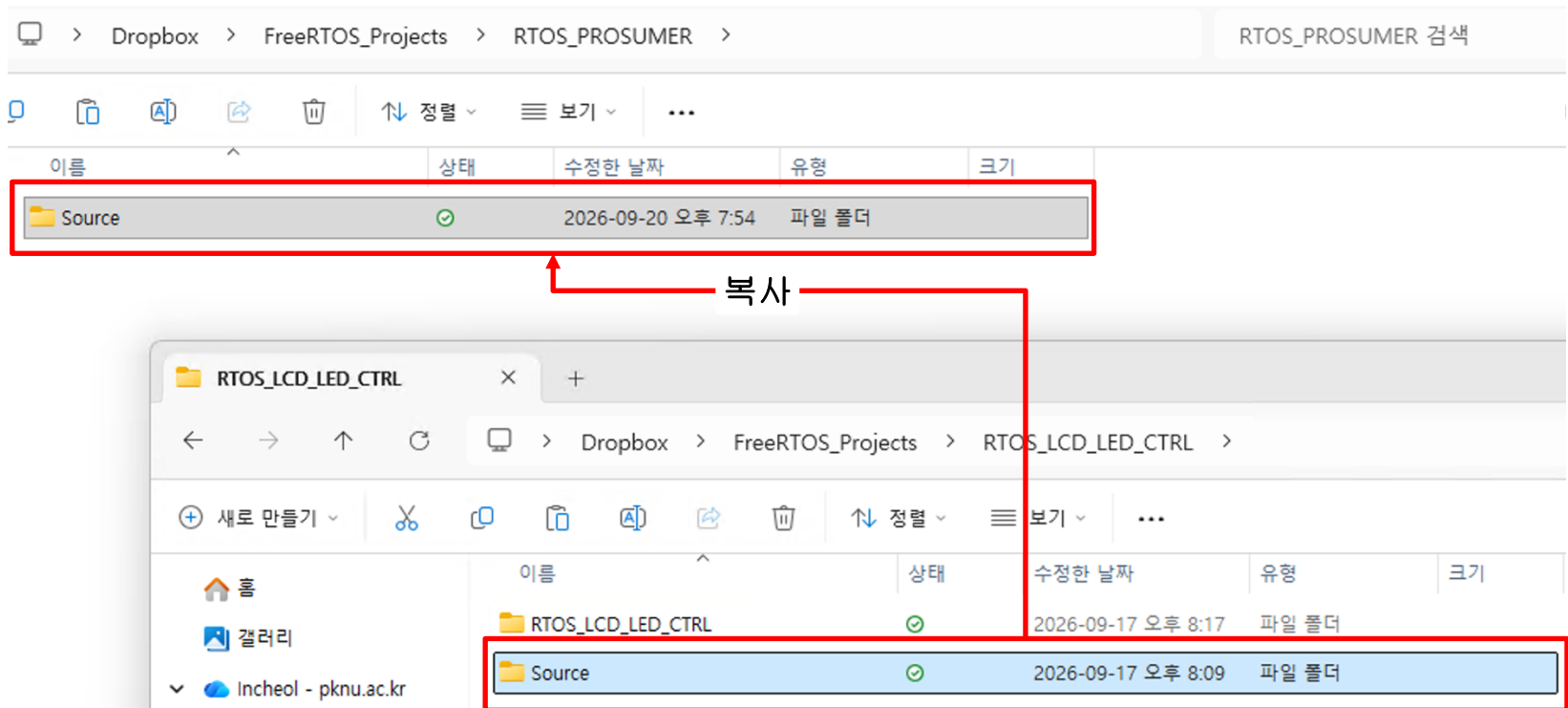
○ 'FreeRTOS_Projects' 디렉터리 내 'RTOS_PROSUMER' 디렉터리 생성

🖥️ > Dropbox > FreeRTOS_Projects >					
📄 📁 📄 📄 🗑️ ⬆️ 정렬 ▾ ≡ 보기 ▾ ⋮					
이름	상태	수정한 날짜	유형	크기	
📁 LCD_DEMO	✔️	2026-09-17 오후 5:41	파일 폴더		
📁 LED_DEMO	✔️	2026-09-17 오후 7:37	파일 폴더		
📁 LED_TOGGLE	✔️	2026-09-17 오후 5:41	파일 폴더		
📁 RTOS_LCD_DEMO	✔️	2026-09-17 오후 6:14	파일 폴더		
📁 RTOS_LCD_LED_CTRL	✔️	2026-09-17 오후 8:17	파일 폴더		
📁 RTOS_LED_DEMO	✔️	2026-09-17 오후 7:32	파일 폴더		
📁 RTOS_PROSUMER	✔️	2026-09-20 오후 6:33	파일 폴더		

FreeRTOS PROSUMER 실습

□ Source 디렉터리 복사

- 기존에 생성 및 사용한 'RTOS_LCD_LED_CTRL\Source' 디렉터리를 'RTOS_PROSUMER' 디렉터리에 복사

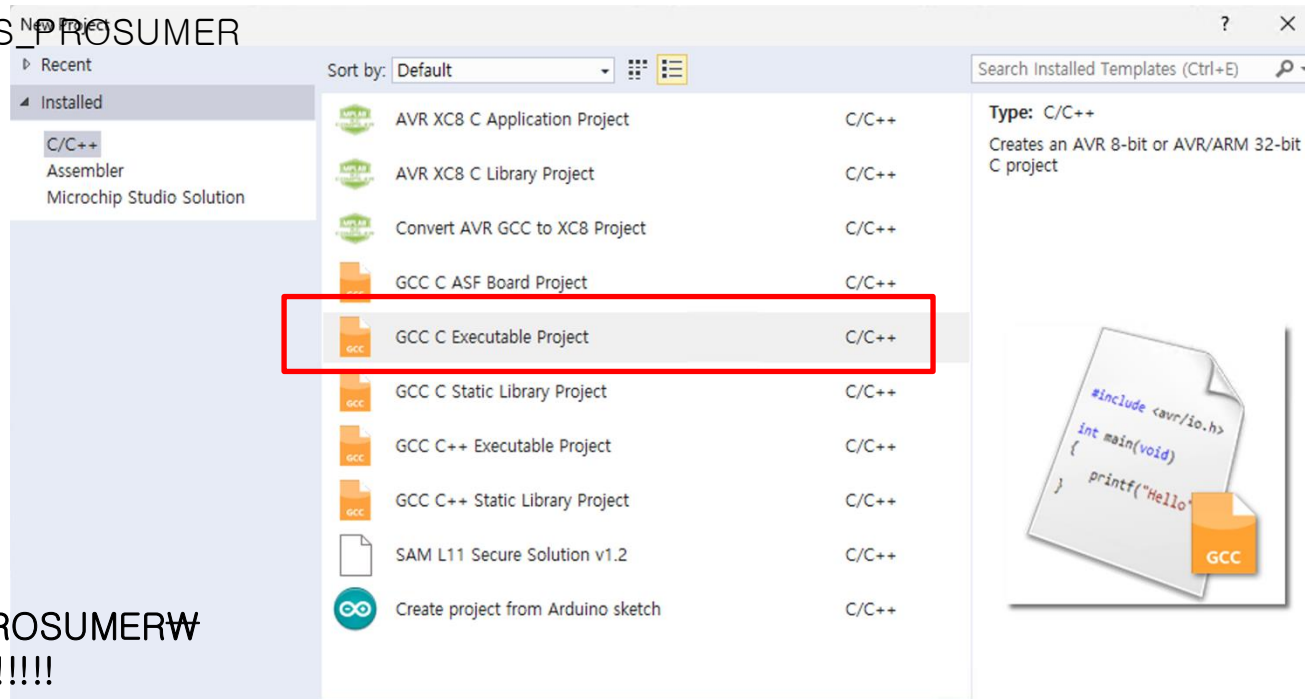


FreeRTOS PROSUMER 실습

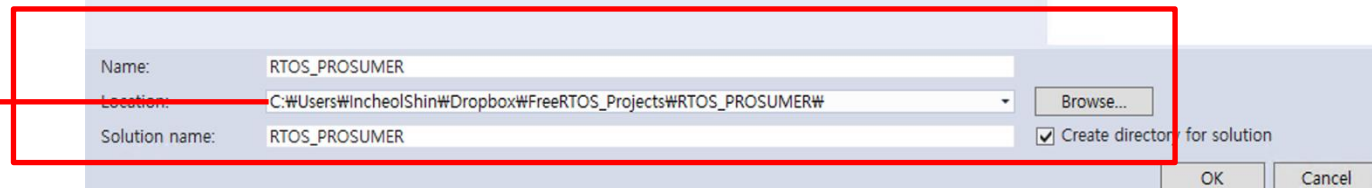
□ RTOS_PROSUMER 프로젝트 생성

○ Microchip Studio 프로그램 실행 후

- GCC C Executable Project 선택
- Name : RTOS_PROSUMER
- Location: FreeRTOS_Projects\RTOS_PROSUMER\
- Solution name: RTOS_PROSUMER

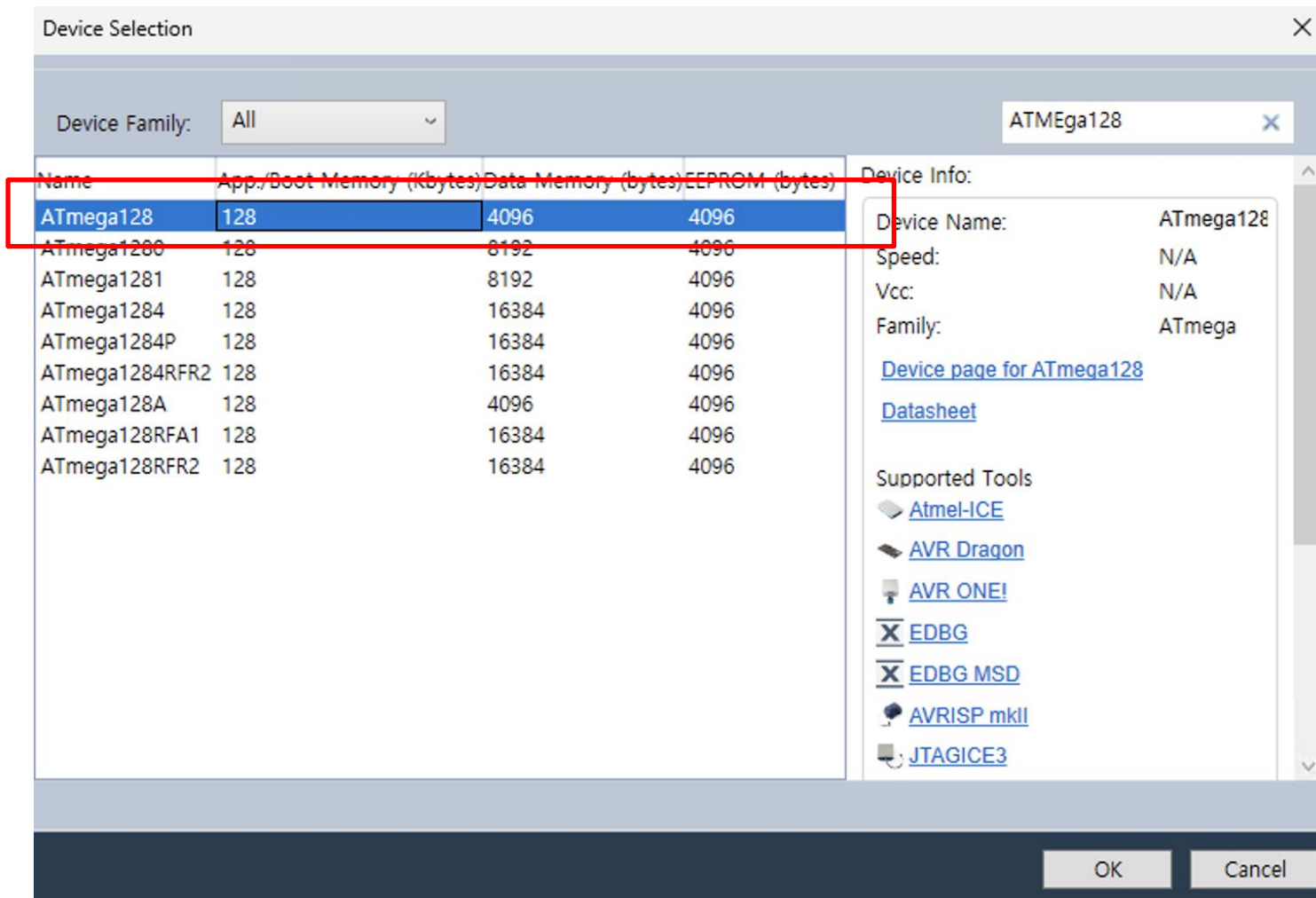


FreeRTOS_Projects\RTOS_PROSUMER\
디렉터리 반드시 확인!!!!!!



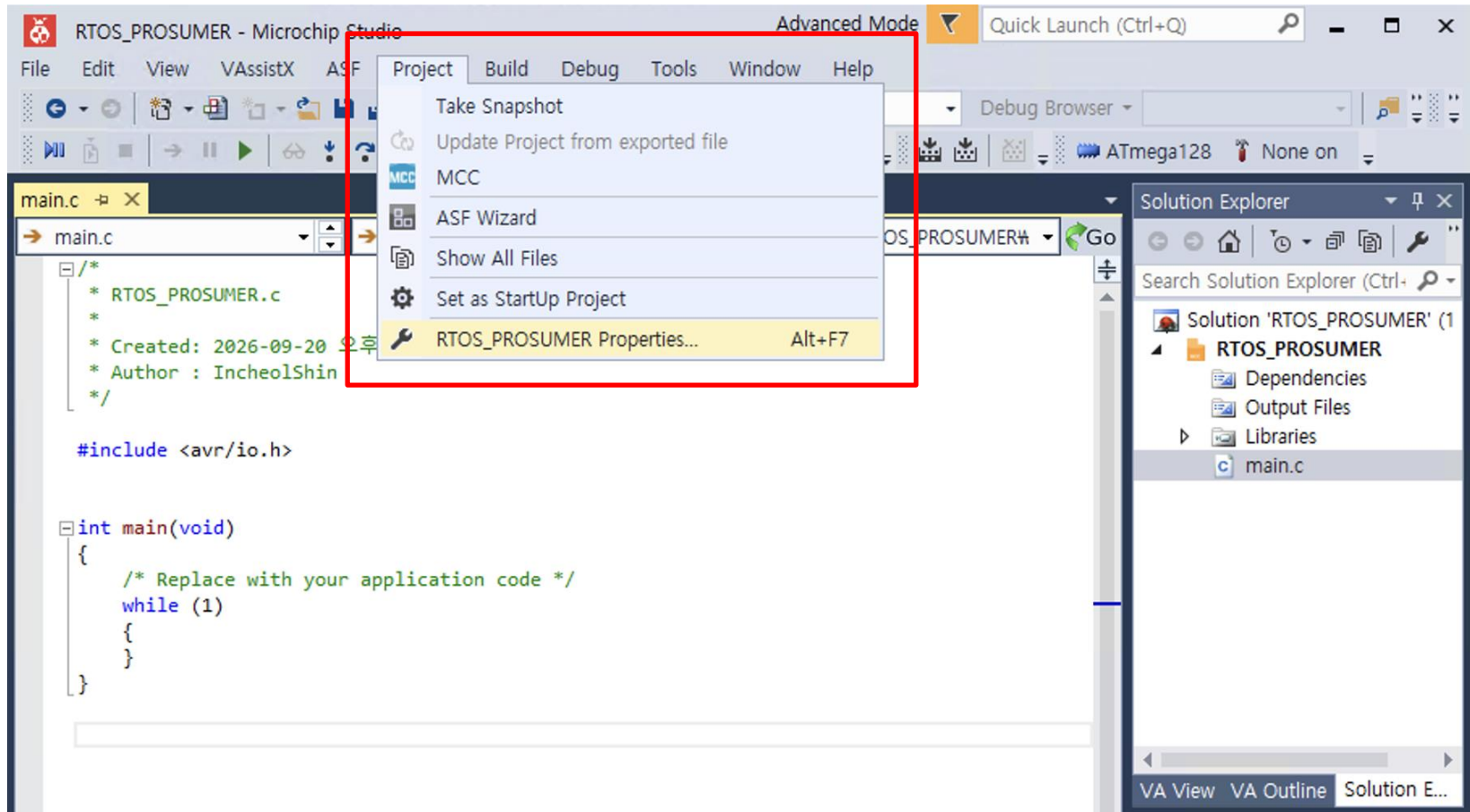
FreeRTOS PROSUMER 실습

- RTOS_PROSUMER 프로젝트 생성 (계속)
- Microchip Studio 프로그램 실행 후 (계속)
 - ATmega128 선택 후 'OK'



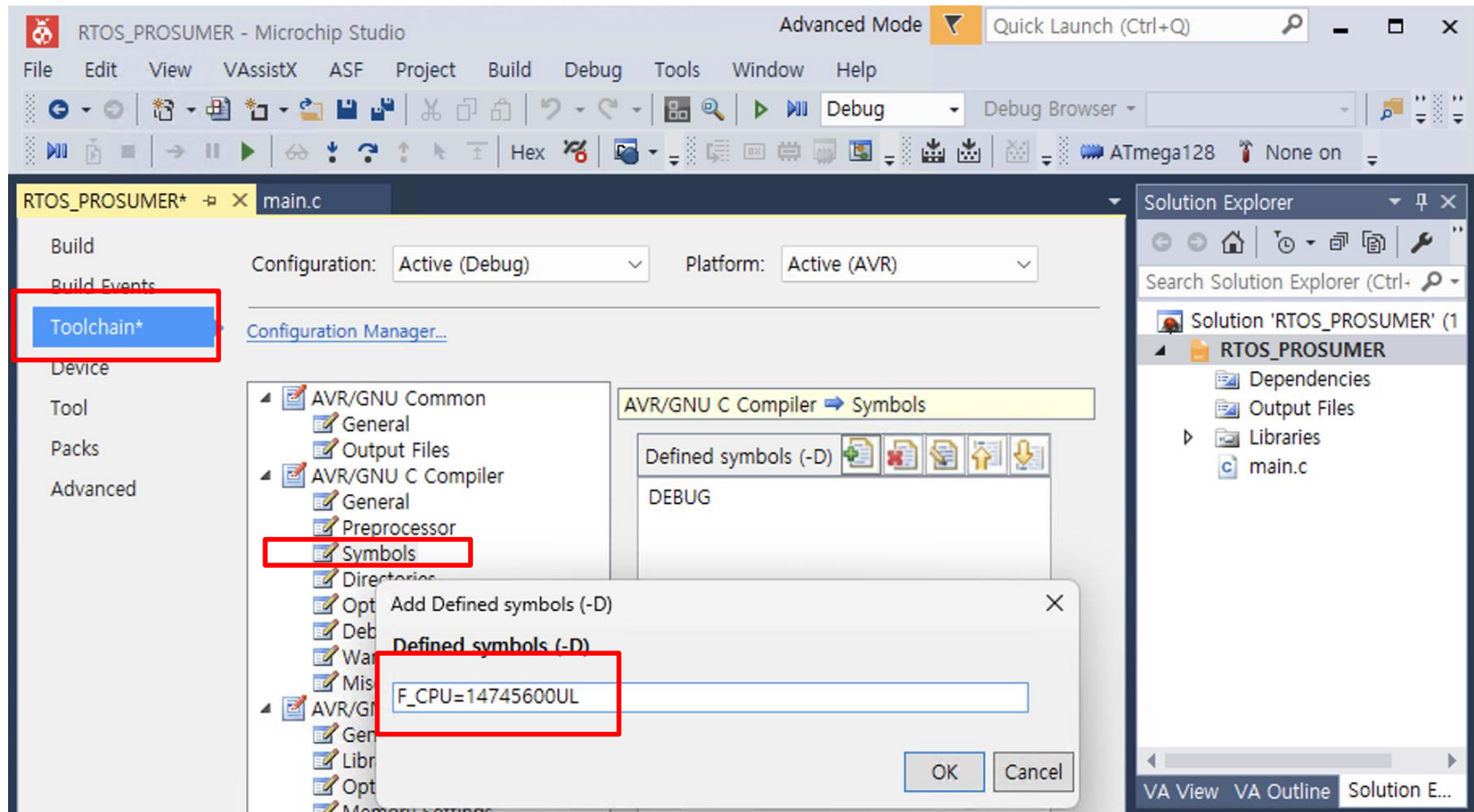
FreeRTOS PROSUMER 실습

- 크로스 컴파일러 심볼 등록
- 프로젝트 속성 설정 선택



FreeRTOS PROSUMER 실습

- 크로스 컴파일러 심볼 등록 (계속)
 - 심볼(F_CPU=14745600UL)

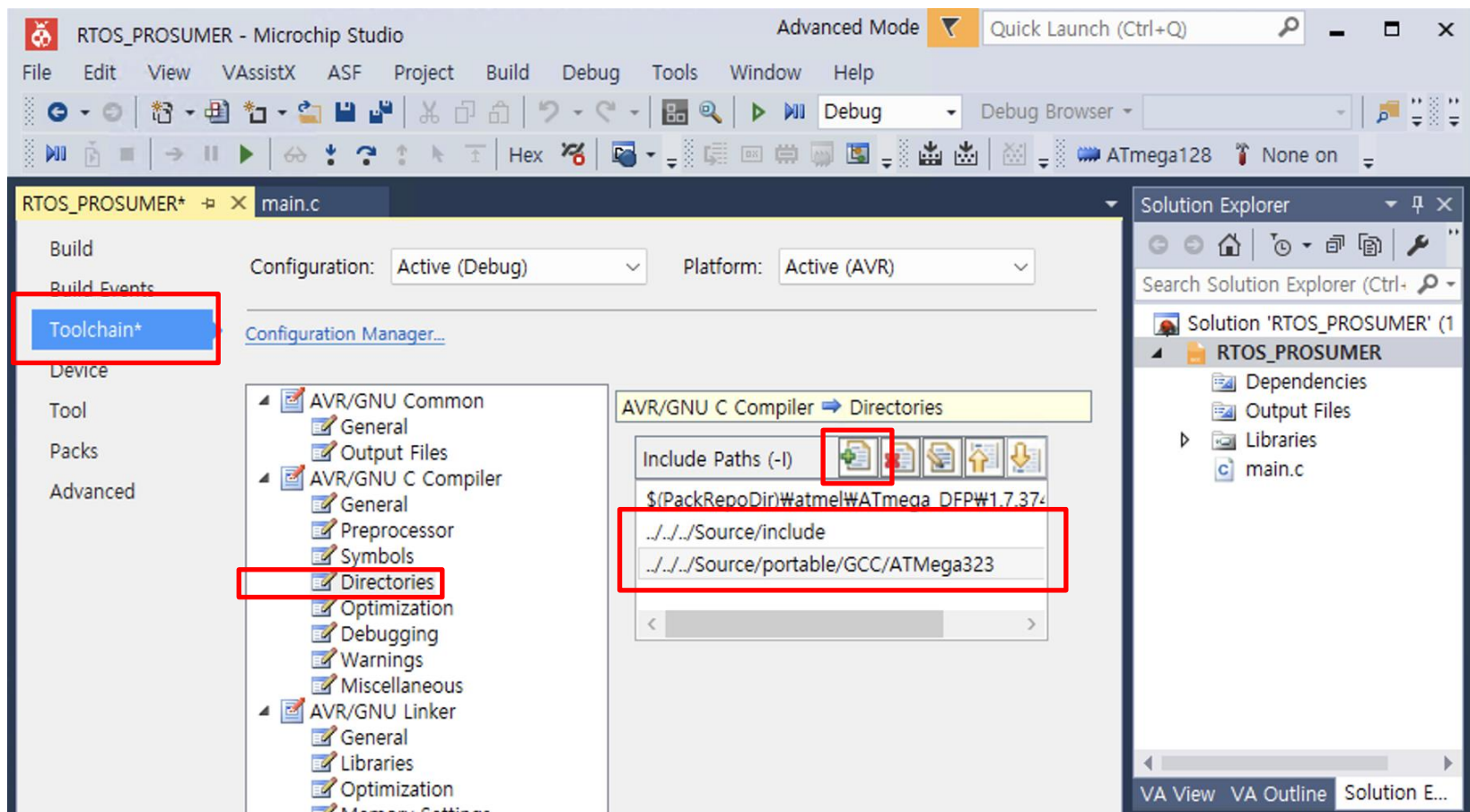


- 반드시 저장(ctrl+s)

FreeRTOS PROSUMER 실습

□ 크로스 컴파일러 디렉터리 등록

- 'RTOS_PROSUMER\Source\Wininclude'
- 'RTOS_PROSUMER\Source\portable\GCC\ATMega323'



➤ 반드시 저장(ctrl+s)

FreeRTOS PROSUMER 실습

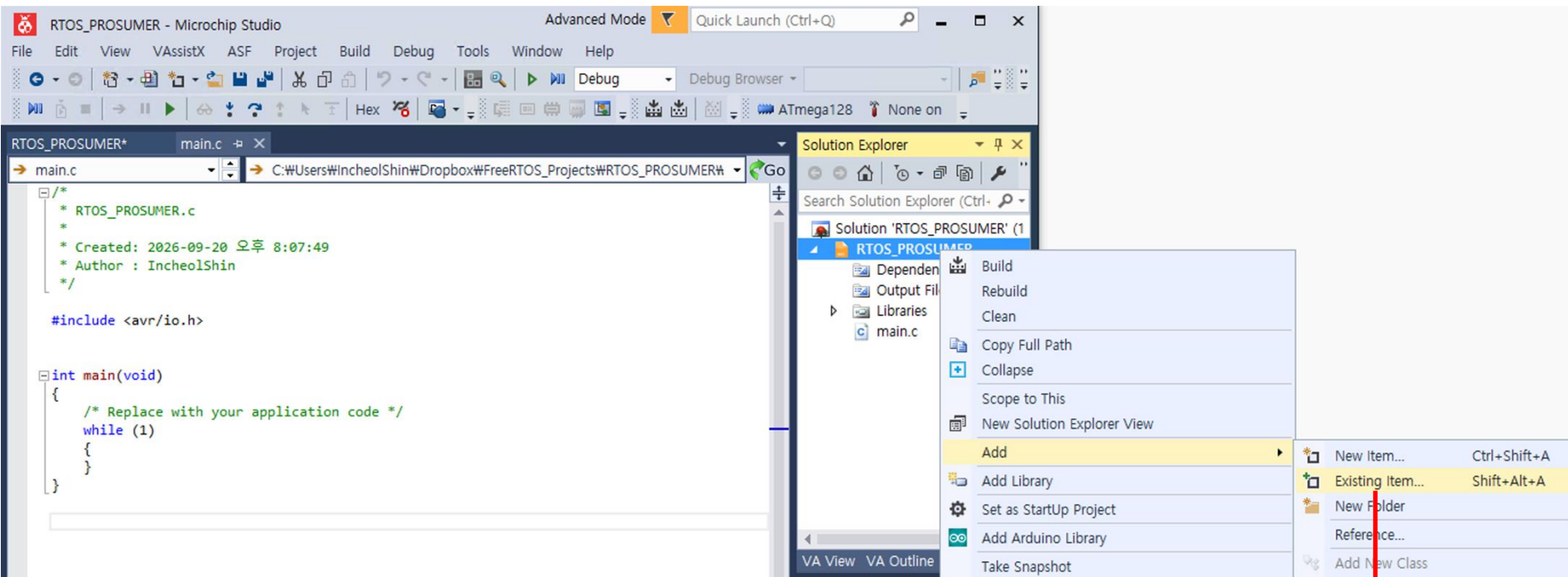
□ 기타 RTOS 소스코드 추가

○ 'RTOS_PROSUMER\Source\W' 내

➤ 'heap_1.c', 'list.c', 'queue.c', 'tasks.c', 'lcd.c' 추가

○ 'RTOS_PROSUMER\Source\Wportable\WGCC\WATMega323\W' 내

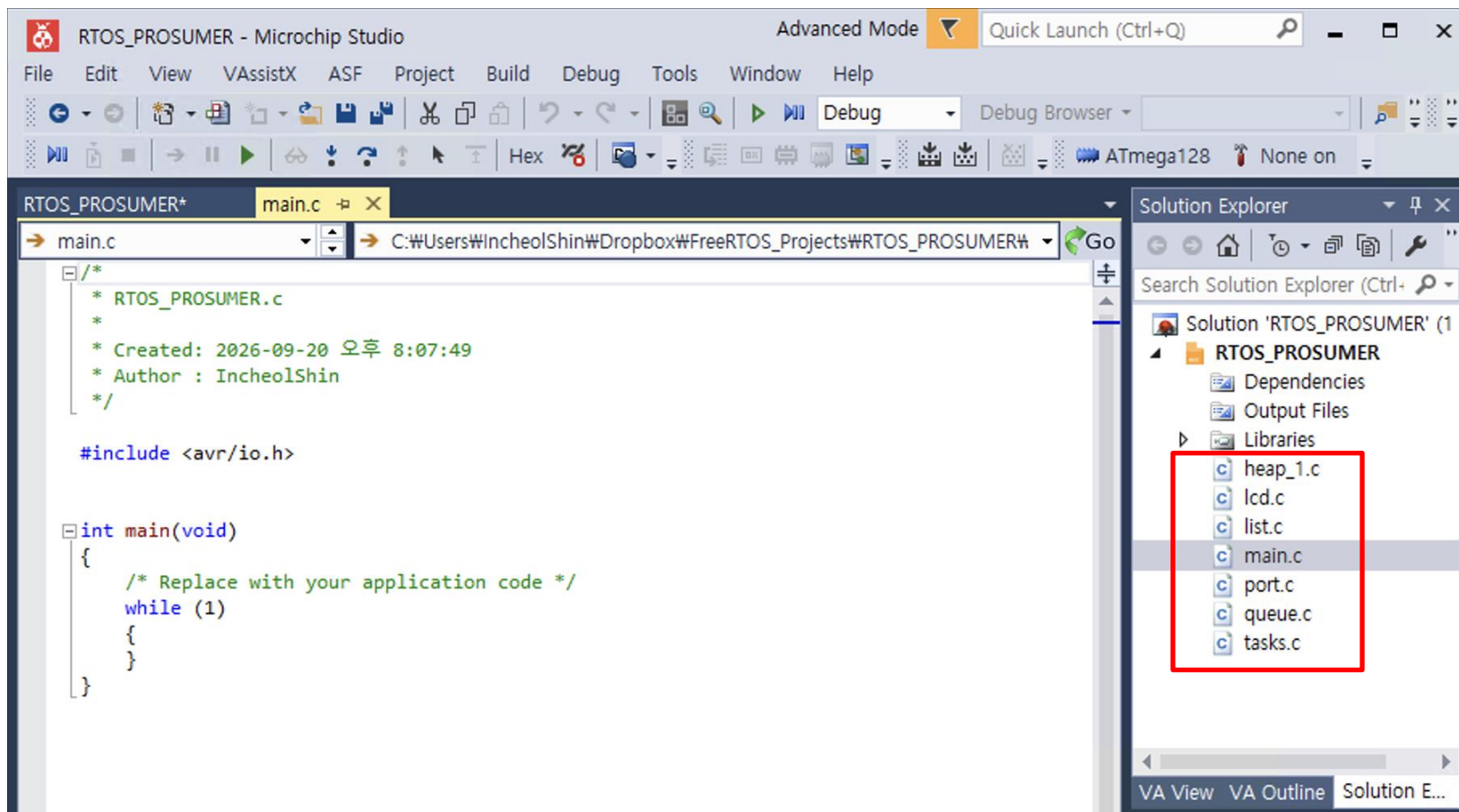
➤ 'port.c' 추가



FreeRTOS_Projects\RTOS_PROSUMER\Source\W 내
기존 파일을 포함하기 때문에 'Existing Item' 선택!!

FreeRTOS PROSUMER 실습

□ 기타 RTOS 소스코드 추가 결과



Producer + Consumer Problem

생산자 소비자 문제

□ Producer + Consumer Problem 개요

○ 한정 버퍼 문제(bounded-buffer problem)

- 생산자-소비자 문제(producer-consumer problem) 는 여러 개의 프로세스를 어떻게 동기화 할 것인가에 관한 고전적인 문제이다.
- 유한한 개수의 물건(데이터)을 임시로 보관하는 보관함(버퍼)에 여러 명의 생산자들과 소비자 들이 접근한다.
 - 생산자는 물건이 하나 만들어지면 그 공간에 저장한다. 이때 저장할 공간이 없는 문제가 발생할 수 있다.
 - 소비자는 물건이 필요할 때 보관함에서 물건을 하나 가져온다. 이 때는 소비할 물건이 없는 문제가 발생할 수 있다.

□ 해결 방법

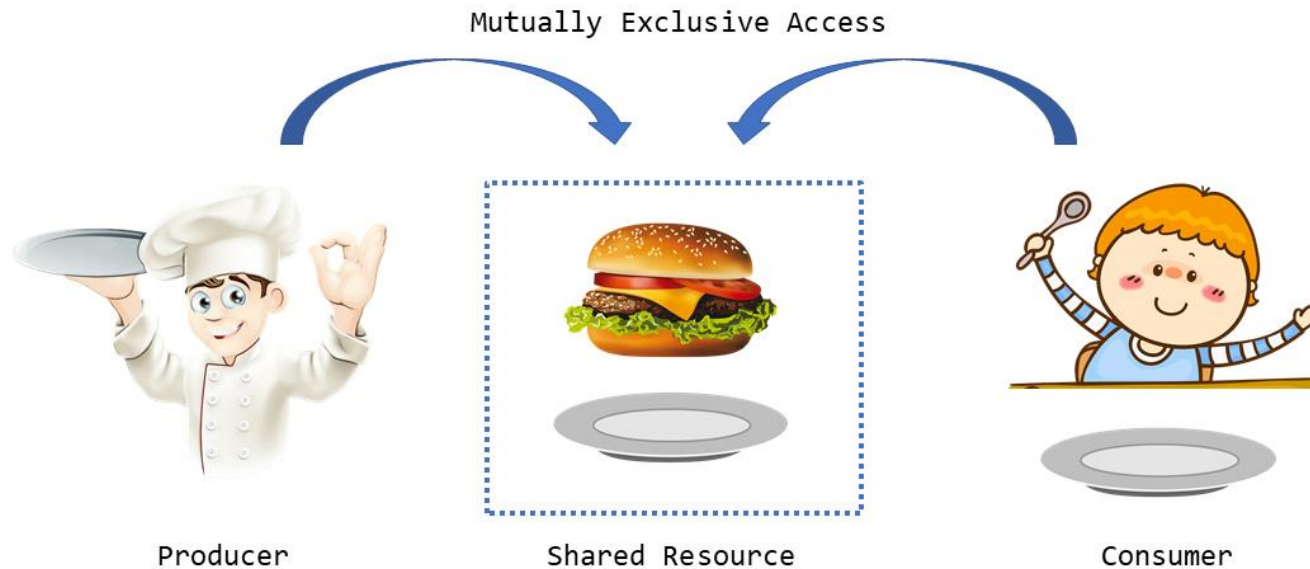
○ 주요 변수

- Empty : 버퍼 내에 저장할 공간이 있는지를 나타낸다. (초기값은 n)
- Full : 버퍼 내에 소비할 아이템이 있는지를 나타낸다. (초기값은 0)
- Mutex : 버퍼에 대한 접근을 통제한다. (초기값은 1)

생산자 소비자 문제

□ Producer + Consumer Problem 개요

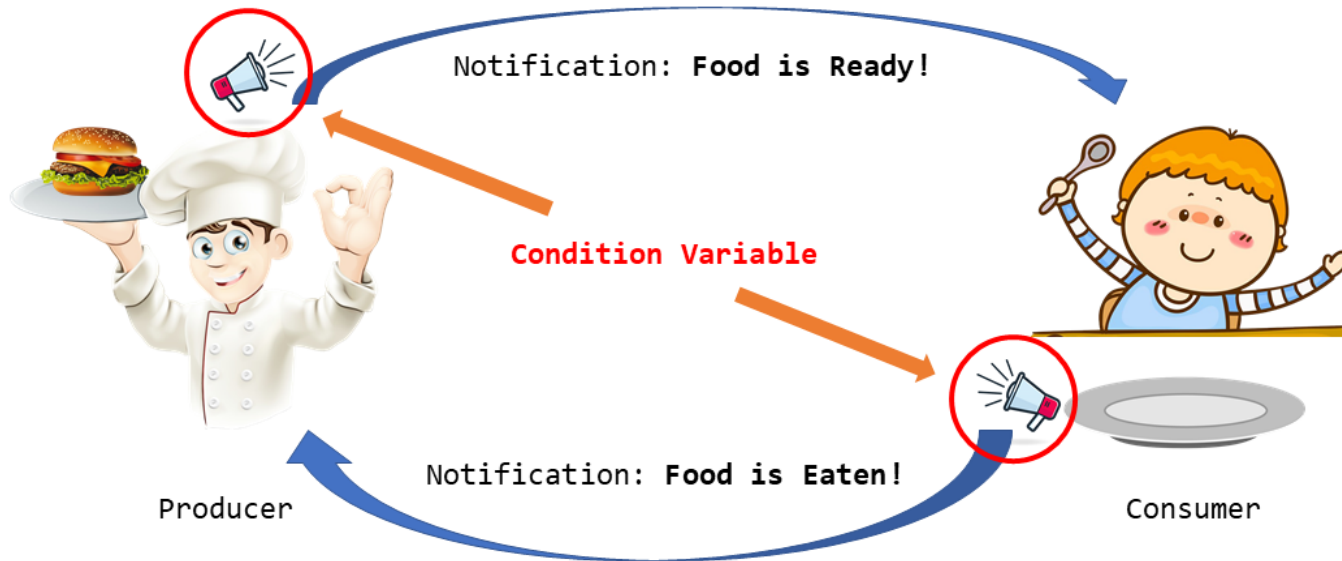
- Semaphore를 이용한 방법



생산자 소비자 문제

□ Producer + Consumer Problem 개요

○ Monitor를 이용한 방법



생산자 소비자 문제

□ 해결 방법(계속)

○ 생산자 프로세스

```
do {  
    ...  
    아이템을 생산한다.  
    ...  
    wait(empty);    //버퍼에 빈 공간이 생길 때까지 기다린다.  
    wait(mutex);    //임계 구역에 진입할 수 있을 때까지 기다린다.  
    ...  
    아이템을 버퍼에 추가한다.  
    ...  
    signal(mutex); //임계 구역을 빠져나왔다고 알려준다.  
    signal(full);   //버퍼에 아이템이 있다고 알려준다.  
} while (1);
```

생산자 소비자 문제

□ 해결 방법(계속)

○ 소비자 프로세스

```
do {  
    wait(full);          //버퍼에 아이템이 생길 때까지 기다린다.  
    wait(mutex);  
    ...  
    버퍼로부터 아이템을 가져온다.  
    ...  
    signal(mutex);  
    signal(empty);      //버퍼에 빈 공간이 생겼다고 알려준다.  
    ...  
    아이템을 소비한다.  
    ...  
} while (1);
```

생산자 소비자 문제

☐ Lost Update 문제 발생 코드

RTOS_PROSUMER - Microchip Studio Advanced Mode Quick Launch (Ctrl+Q)

File Edit View VAssistX ASF Project Build Debug Tools Window Help

Debug Debug Browser ATmega128 None on

RTOS_PROSUMER main.c*

main.c C:\Users\WincheolShin\Dropbox\FreeRTOS_Projects\RTOS_PROSUMER\

```
#include <avr/io.h>
#include <stdint.h>

#include "FreeRTOS.h"
#include "task.h"
#include "lcd.h"

#define BUFFER_SIZE      8

#define NUM_PRODUCERS    5
#define NUM_CONSUMERS    5

#define LED_ACTIVE_LOW   1

#define PRODUCER_MIN_DELAY 2000
#define PRODUCER_MAX_DELAY 4000

#define CONSUMER_MIN_DELAY 4000
#define CONSUMER_MAX_DELAY 7000

/*
 * 공유 버퍼
 *
 * 세마포어/Mutex를 사용하지 않음
 */
volatile uint8_t buffer[BUFFER_SIZE];

volatile uint8_t head = 0;
volatile uint8_t tail = 0;
volatile uint8_t count = 0;

/* Producer / Consumer ID */
static uint8_t producer_id[NUM_PRODUCERS] = {0, 1, 2, 3, 4};
static uint8_t consumer_id[NUM_CONSUMERS] = {0, 1, 2, 3, 4};

/*
 * 간단한 의사 난수 생성기
 */
static uint16_t random_number(uint16_t *seed) {
    *seed = (*seed * 25173U) + 13849U;
```

83 %

Solution Explorer

Search Solution Explorer (Ctrl+Q)

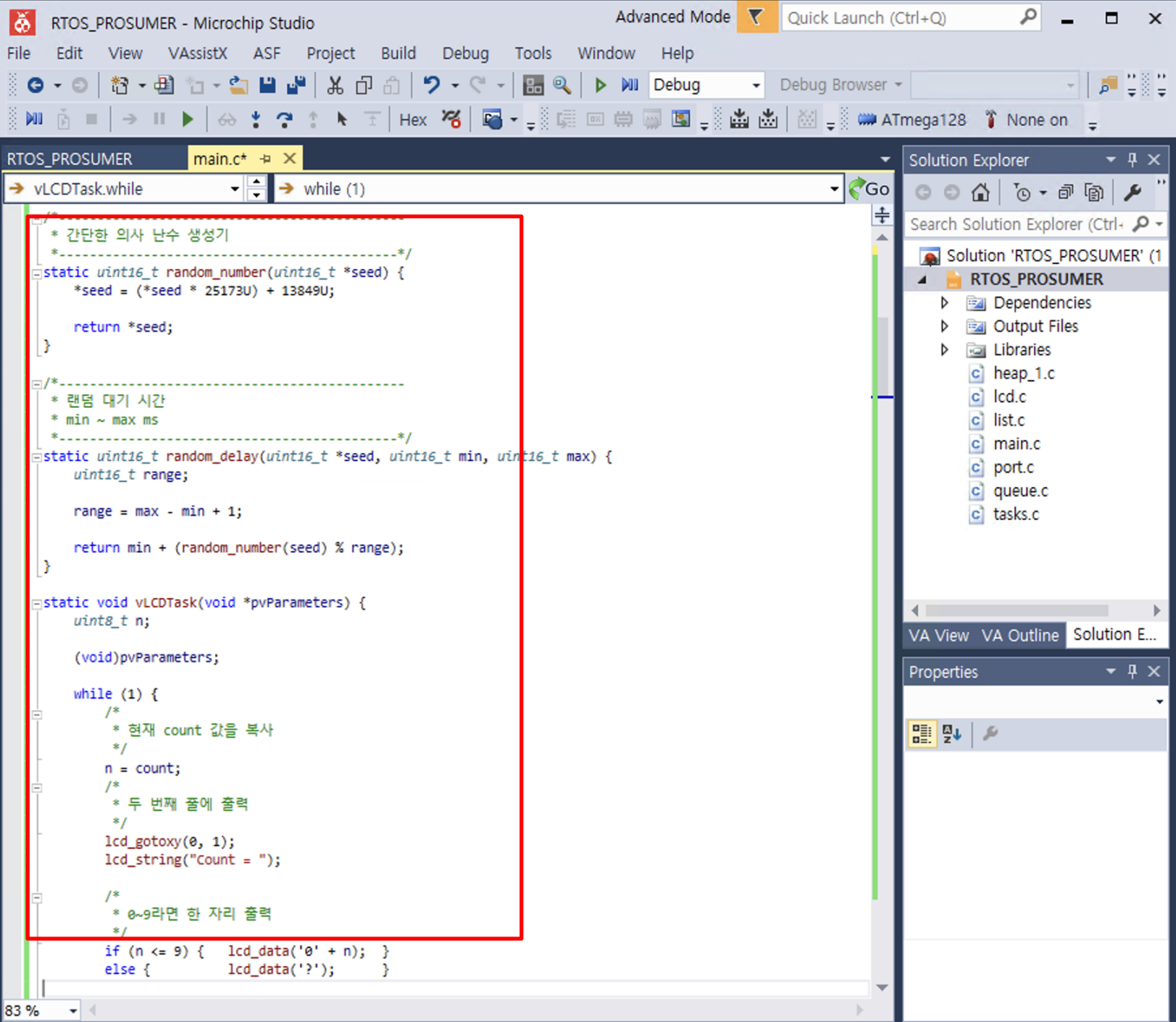
Solution 'RTOS_PROSUMER' (1)

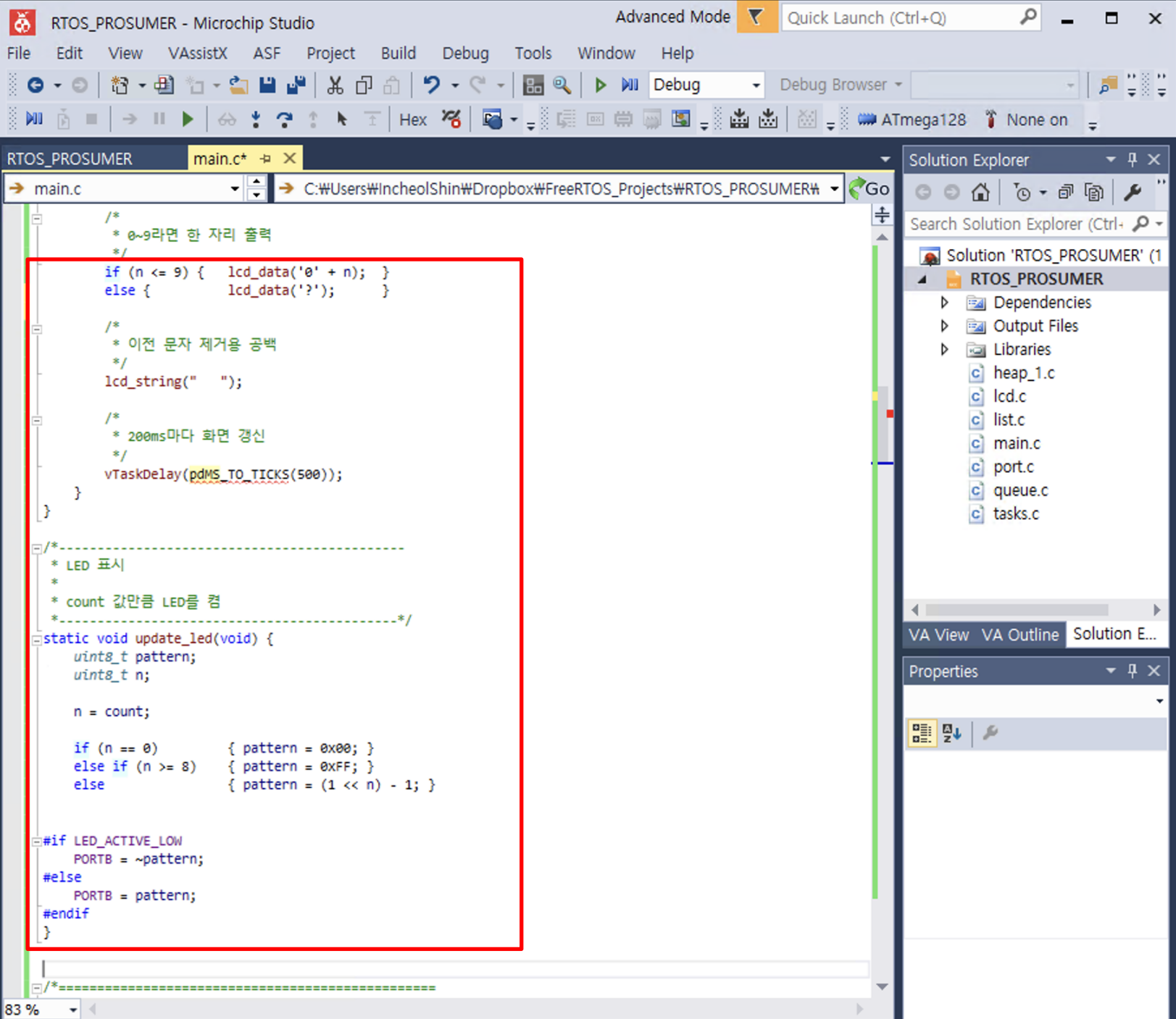
RTOS_PROSUMER

- Dependencies
- Output Files
- Libraries
 - heap_1.c
 - lcd.c
 - list.c
 - main.c
 - port.c
 - queue.c
 - tasks.c

VA View VA Outline Solution E...

Properties





RTOS_PROSUMER - Microchip Studio

Advanced Mode Quick Launch (Ctrl+Q)

File Edit View VAssistX ASF Project Build Debug Tools Window Help

Debug Debug Browser

ATmega128 None on

RTOS_PROSUMER main.c*

main.c C:\Users\WincheolShin\Dropbox\FreeRTOS_Projects\RTOS_PROSUMER\

```
/*-----  
 * Producer  
 *-----*/  
static void vProducerTask(void *pvParameters) {  
    uint8_t id;  
  
    uint8_t data = 0;  
  
    uint8_t temp_count;  
    uint8_t temp_head;  
  
    uint16_t seed;  
    uint16_t delay_time;  
  
    id = *((uint8_t *)pvParameters);  
  
    seed = 1000 + ((uint16_t)id * 3000);  
  
    while (1) {  
        /*-----  
         * 랜덤 시간 대기  
         *-----*/  
        delay_time = random_delay(&seed, PRODUCER_MIN_DELAY, PRODUCER_MAX_DELAY);  
  
        vTaskDelay(pdMS_TO_TICKS(delay_time));  
  
        /*-----  
         * 버퍼가 꽉 찼는지 확인  
         * 문제:  
         * 여러 Producer가 동시에  
         * count < BUFFER_SIZE 를 확인할 수 있음  
         *-----*/  
        if (count < BUFFER_SIZE) { data++;  
            /*-----  
             * head 읽기  
             *-----*/  
            temp_head = head;  
  
            /*  
             * ★ Race Condition을 잘 발생시키기 위해  
             */  
        }  
    }  
}
```

83 %

Solution Explorer

Search Solution Explorer (Ctrl+)

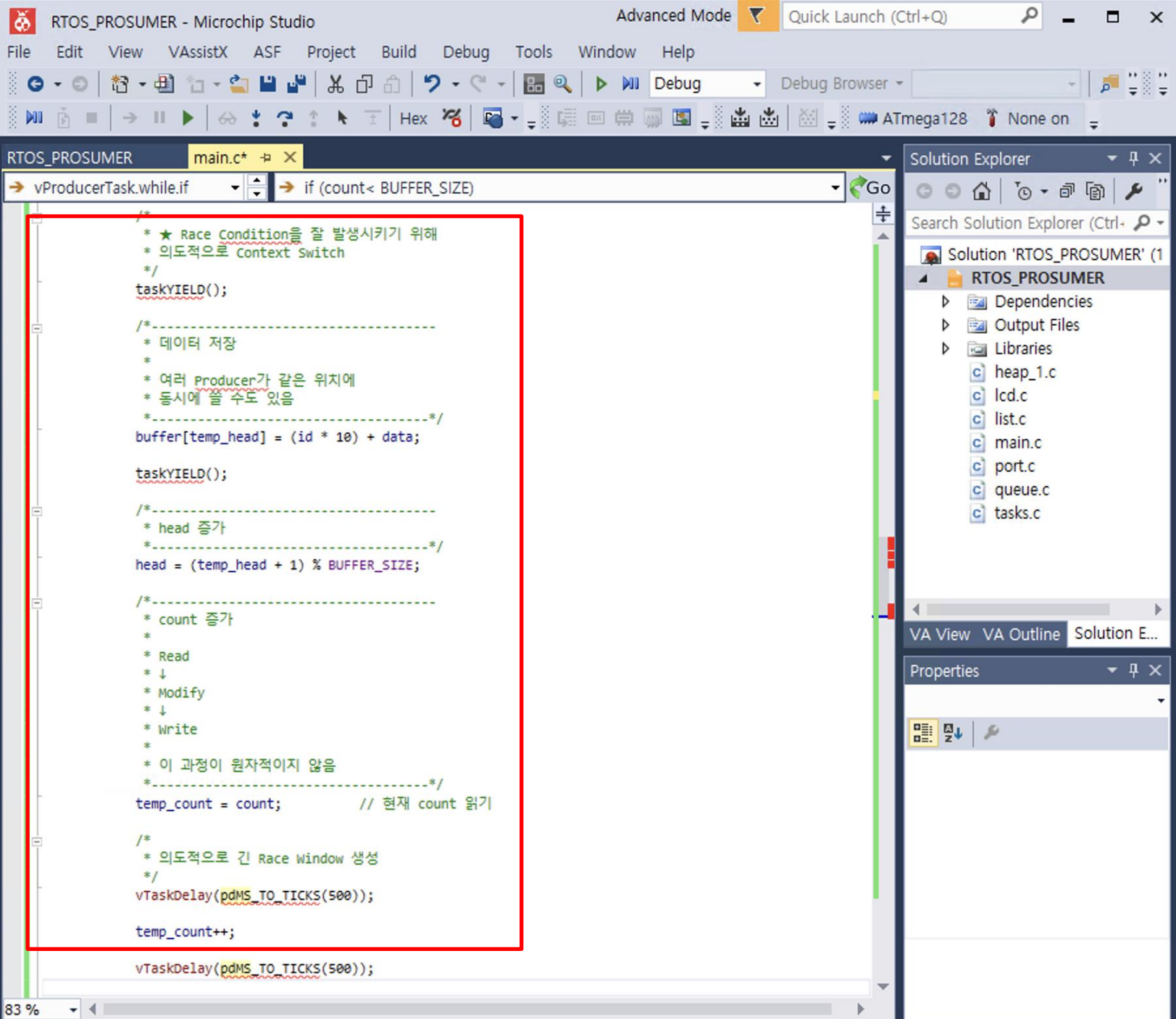
Solution 'RTOS_PROSUMER' (1)

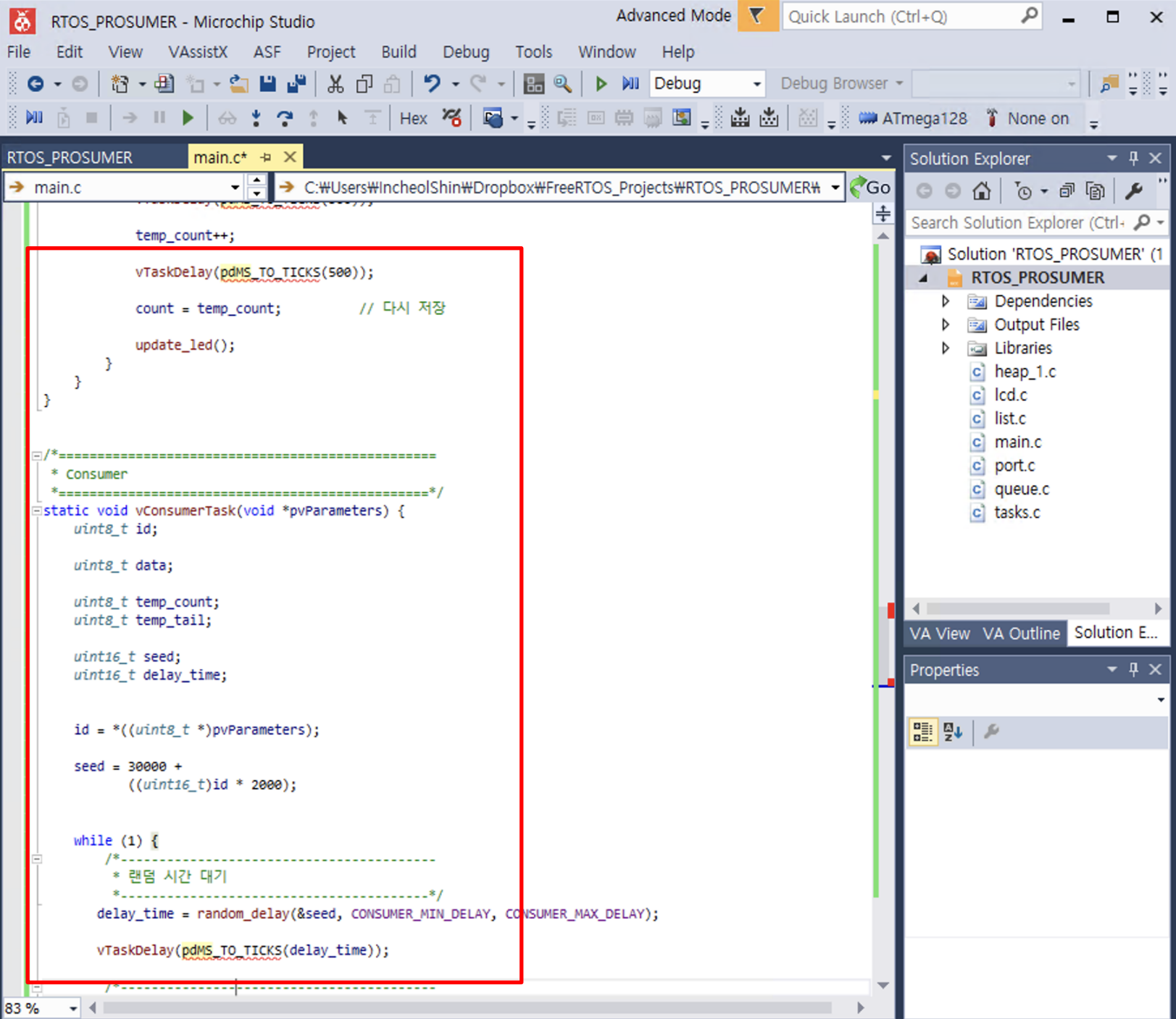
RTOS_PROSUMER

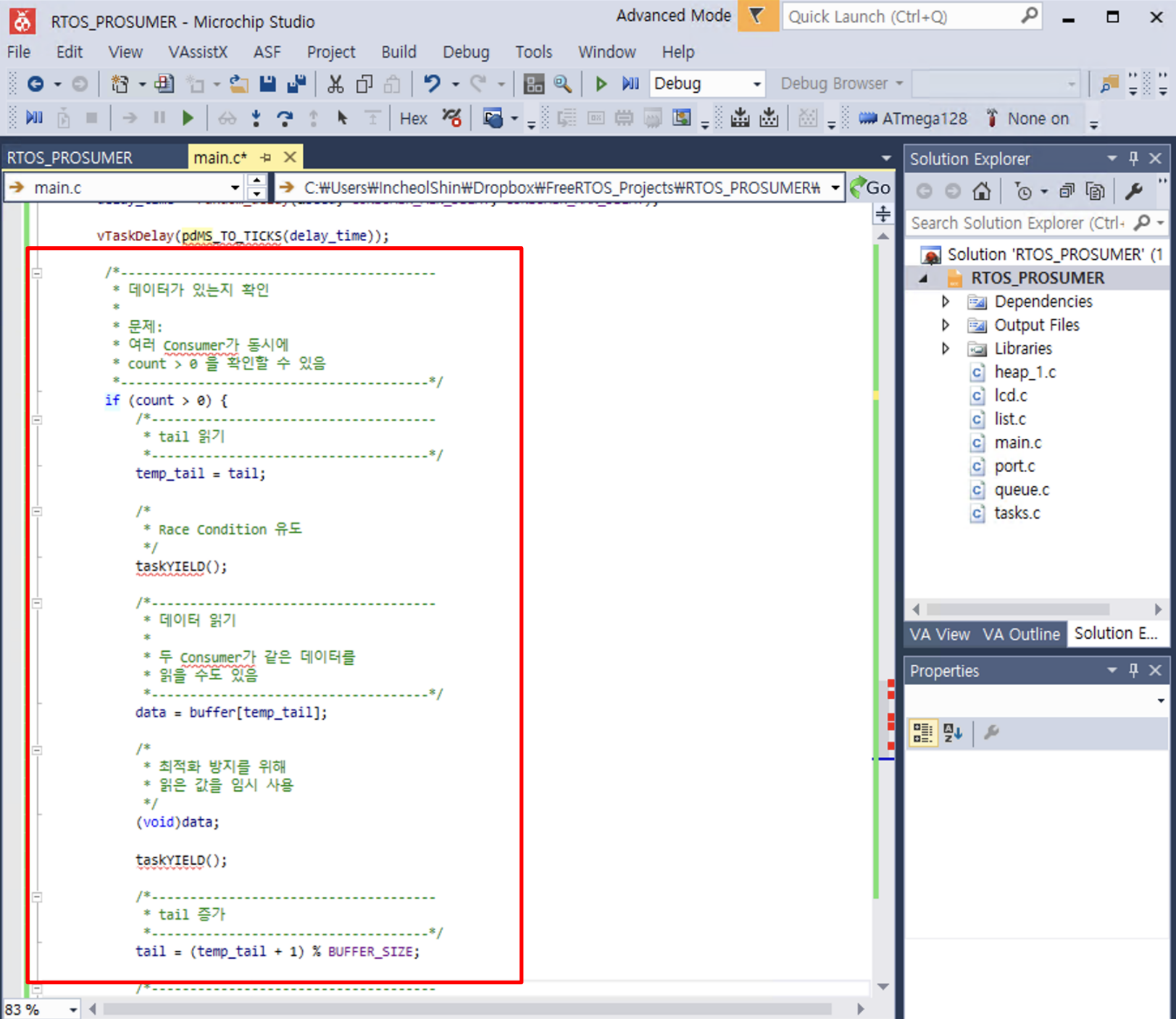
- Dependencies
- Output Files
- Libraries
 - heap_1.c
 - lcd.c
 - list.c
 - main.c
 - port.c
 - queue.c
 - tasks.c

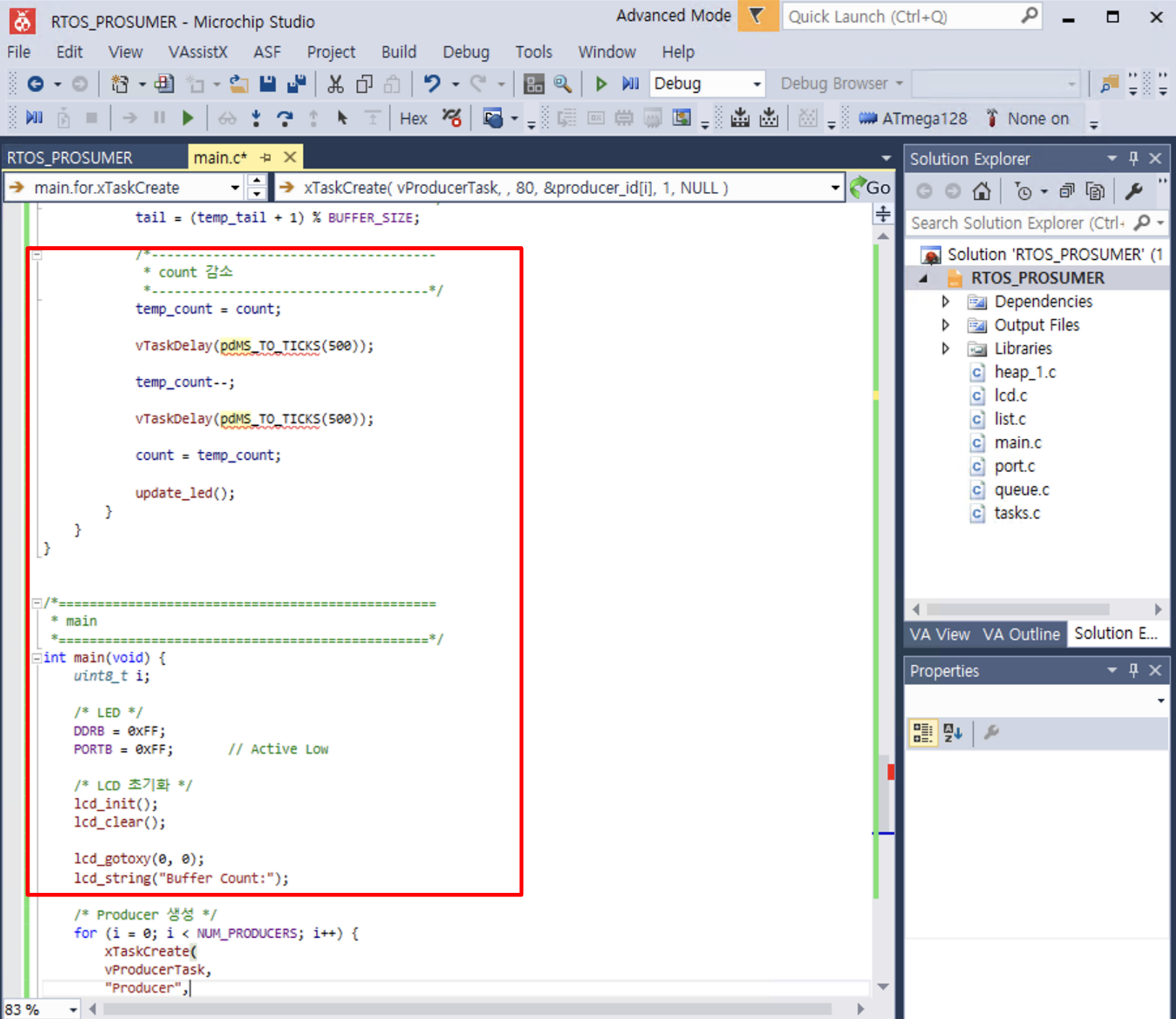
VA View VA Outline Solution E...

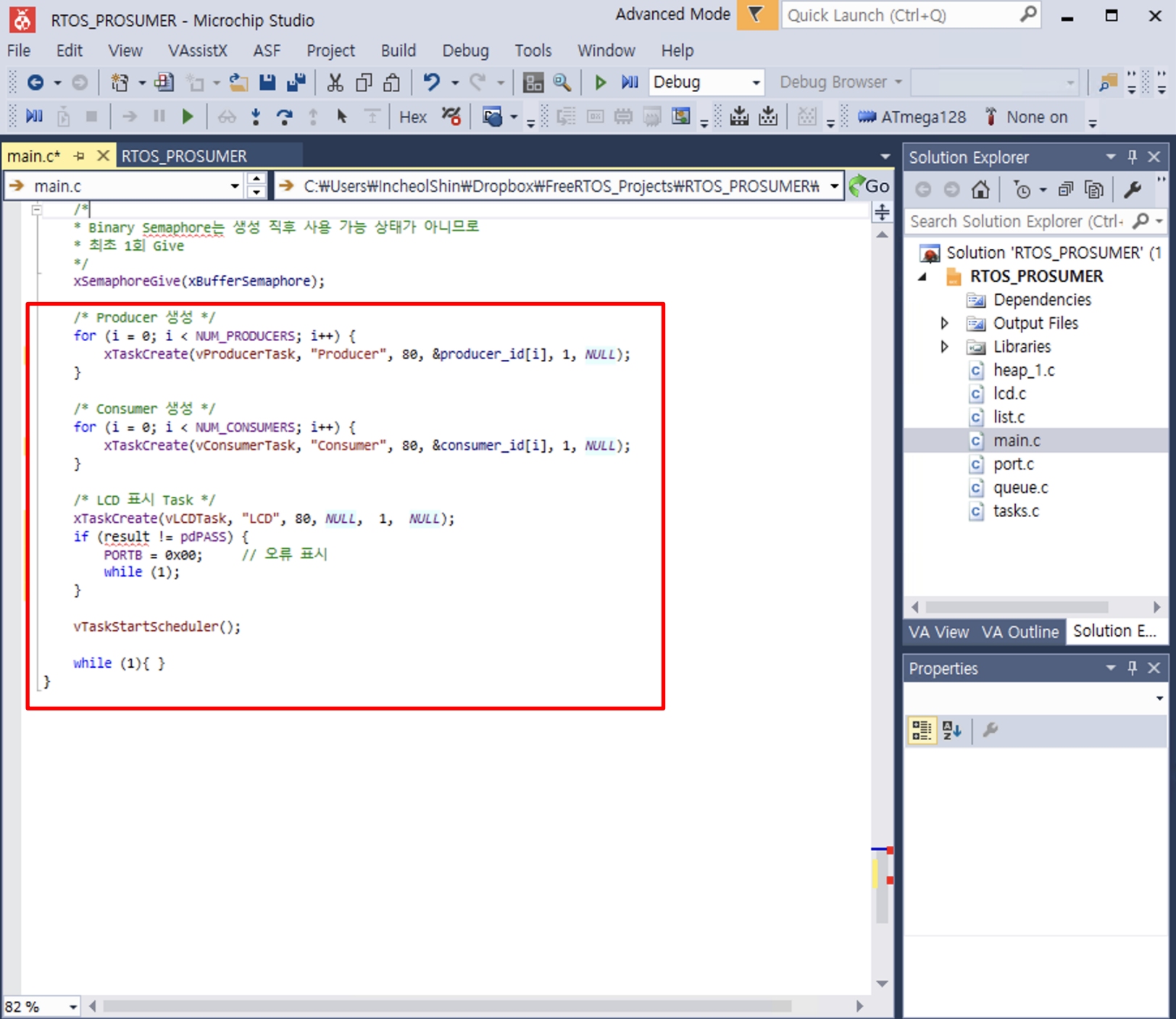
Properties





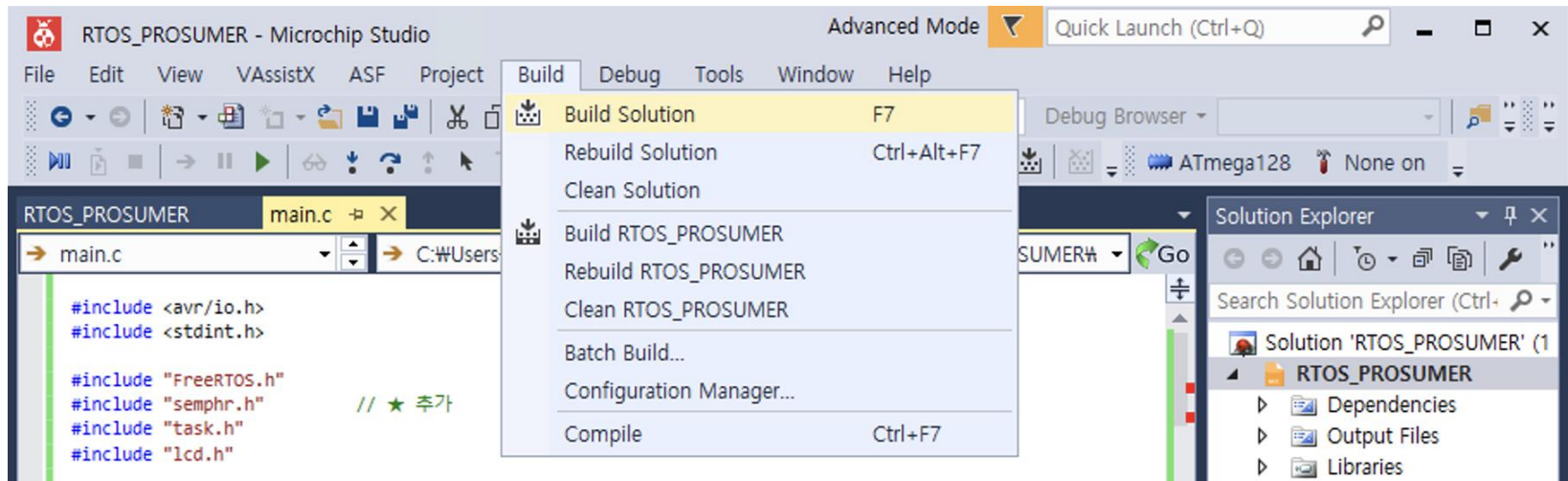




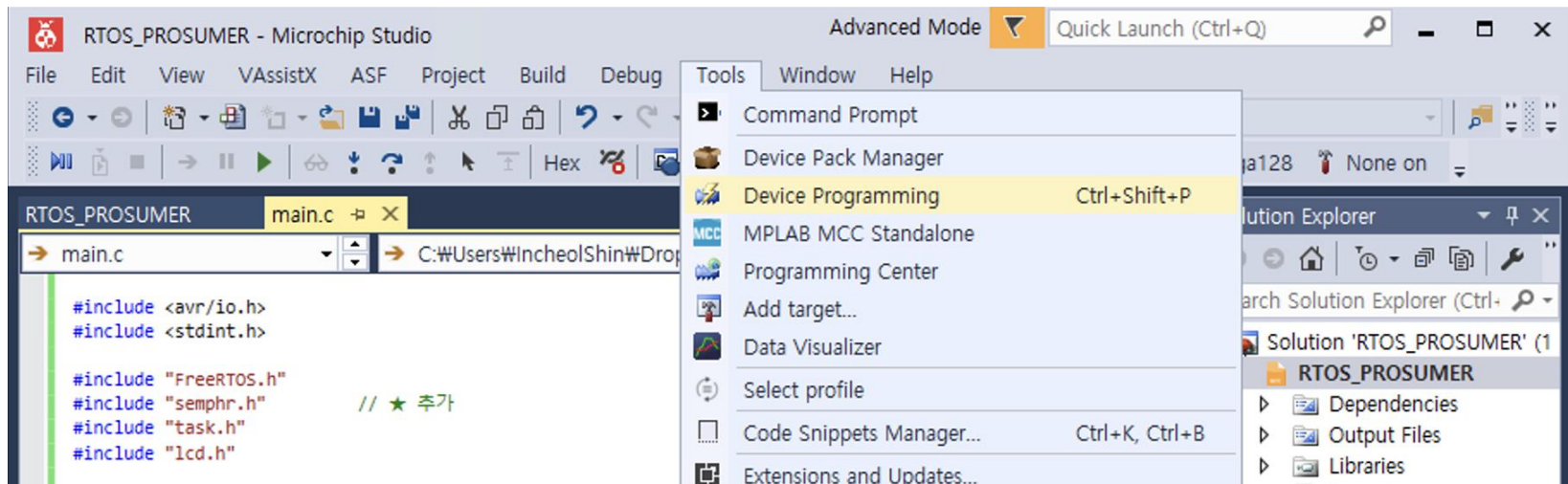


생산자 소비자 문제

□ Build



□ Device Programming

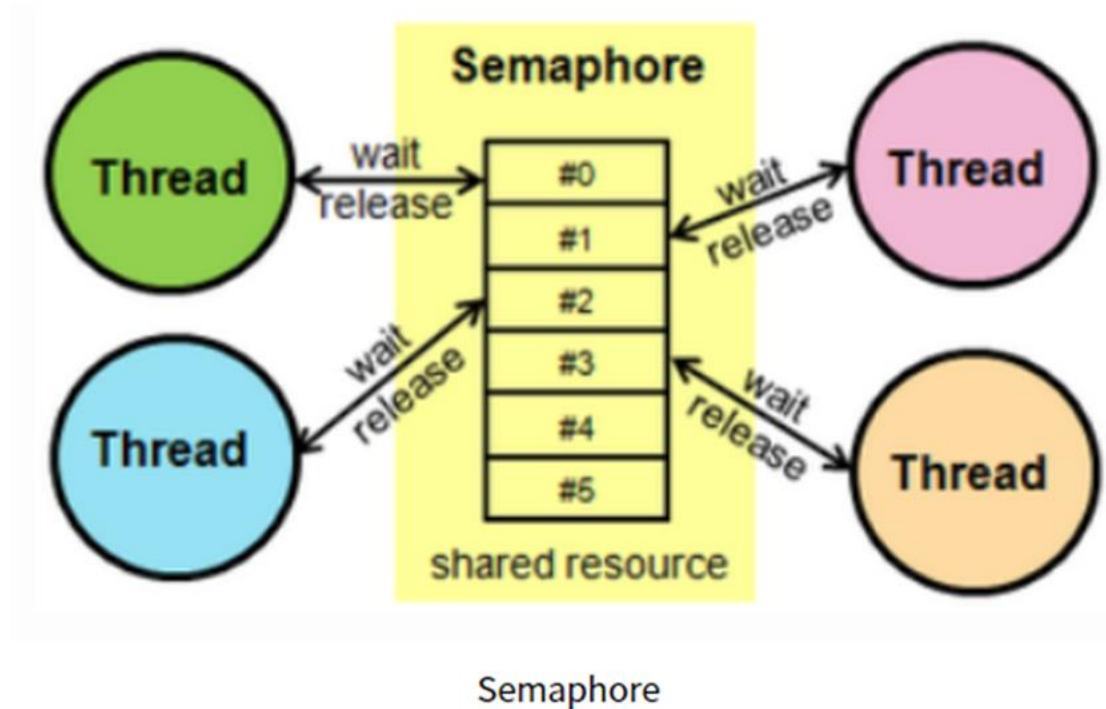


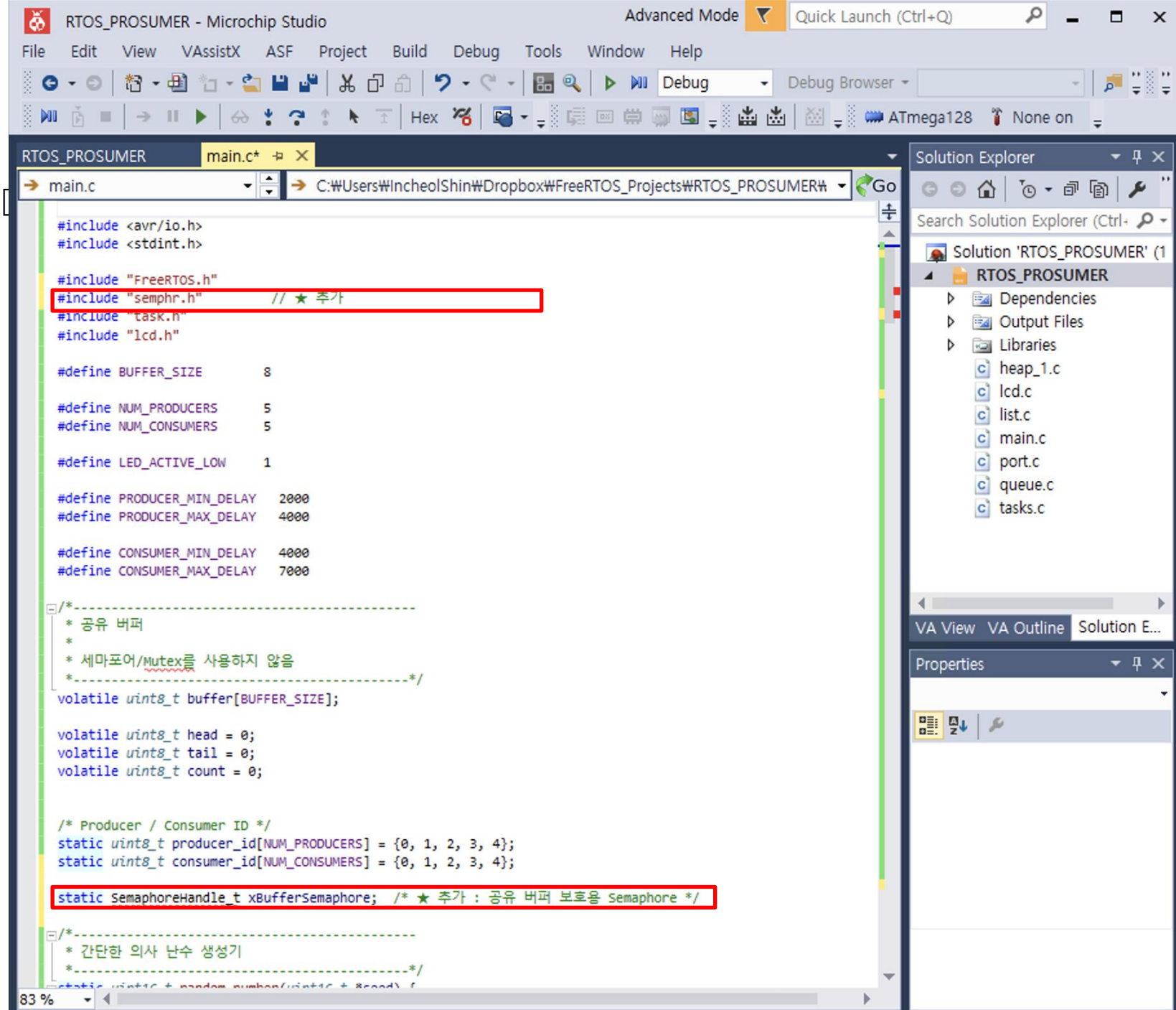
생산자 소비자 문제

☐ Lost Update Problem Occurred!!!!

생산자 소비자 문제

- Lost Update 문제 발생 제거 코드
 - 세마포어 이용





RTOS_PROSUMER - Microchip Studio

File Edit View VAssistX ASF Project Build Debug Tools Window Help

Debug ATmega128 None on

RTOS_PROSUMER main.c*

main.c C:\Users\WincheolShin\Dropbox\FreeRTOS_Projects\RTOS_PROSUMER\

```

}
}

/*=====
 * main
 *=====*/
int main(void) {
    uint8_t i;

    /* LED */
    DDRB = 0xFF;
    PORTB = 0xFF;    // Active Low

    /* LCD 초기화 */
    lcd_init();
    lcd_clear();

    lcd_gotoxy(0, 0);
    lcd_string("Buffer Count:");

    xBufferSemaphore = xSemaphoreCreateBinary();    /* ★ 추가 : Binary Semaphore 생성 */
    if (xBufferSemaphore == NULL) {
        PORTB = 0x00;    // 오류 표시
        while (1);
    }

    /*
     * Binary Semaphore는 생성 직후 사용 가능 상태가 아니므로
     * 최초 1회 Give
     */
    xSemaphoreGive(xBufferSemaphore);

    /* Producer 생성 */
    for (i = 0; i < NUM_PRODUCERS; i++) {
        xTaskCreate(
            vProducerTask,
            "Producer",
            80,
            &producer_id[i],
            1,
            NULL
        );
    }
}

```

83 %

Solution Explorer

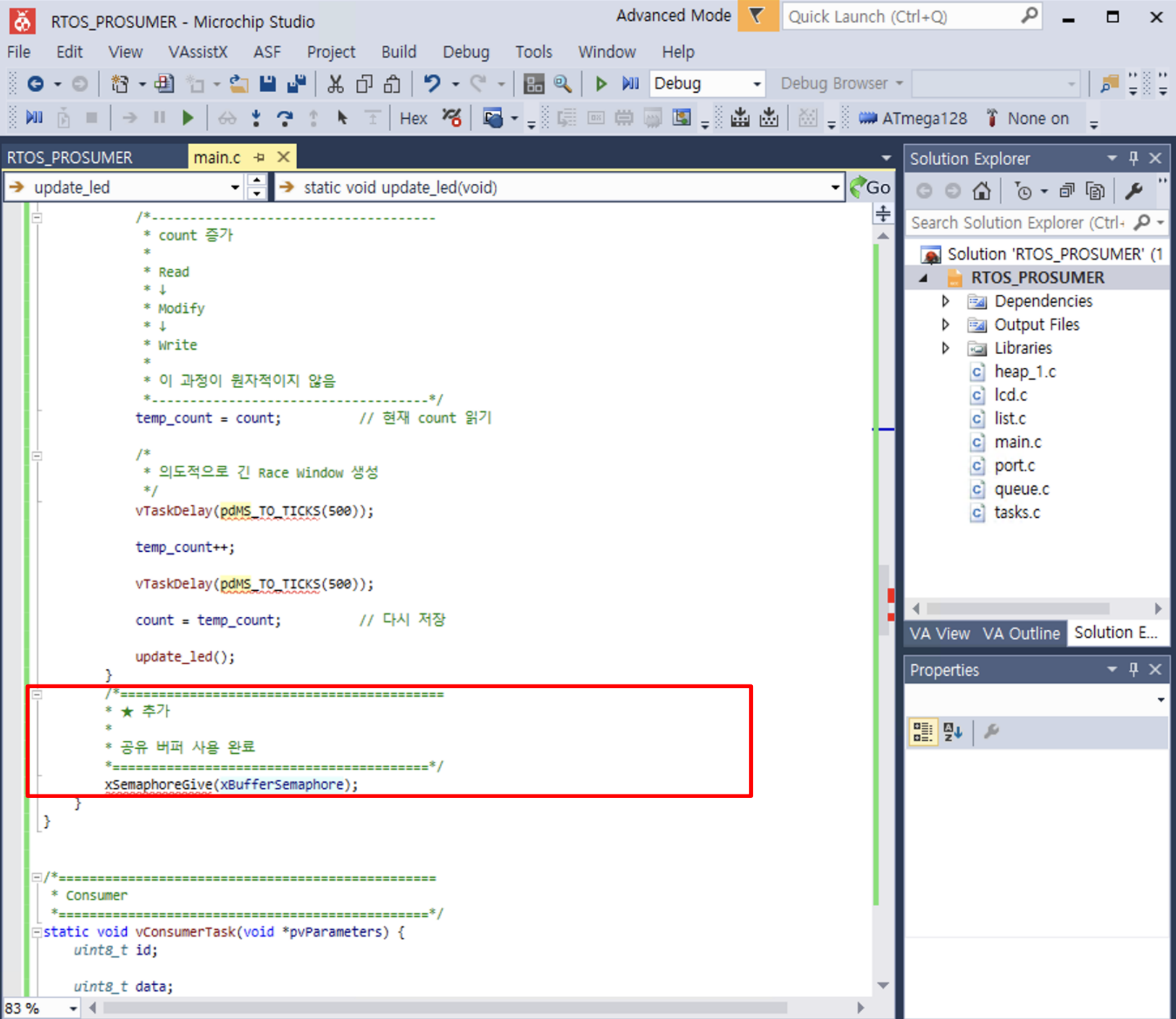
Search Solution Explorer (Ctrl+Q)

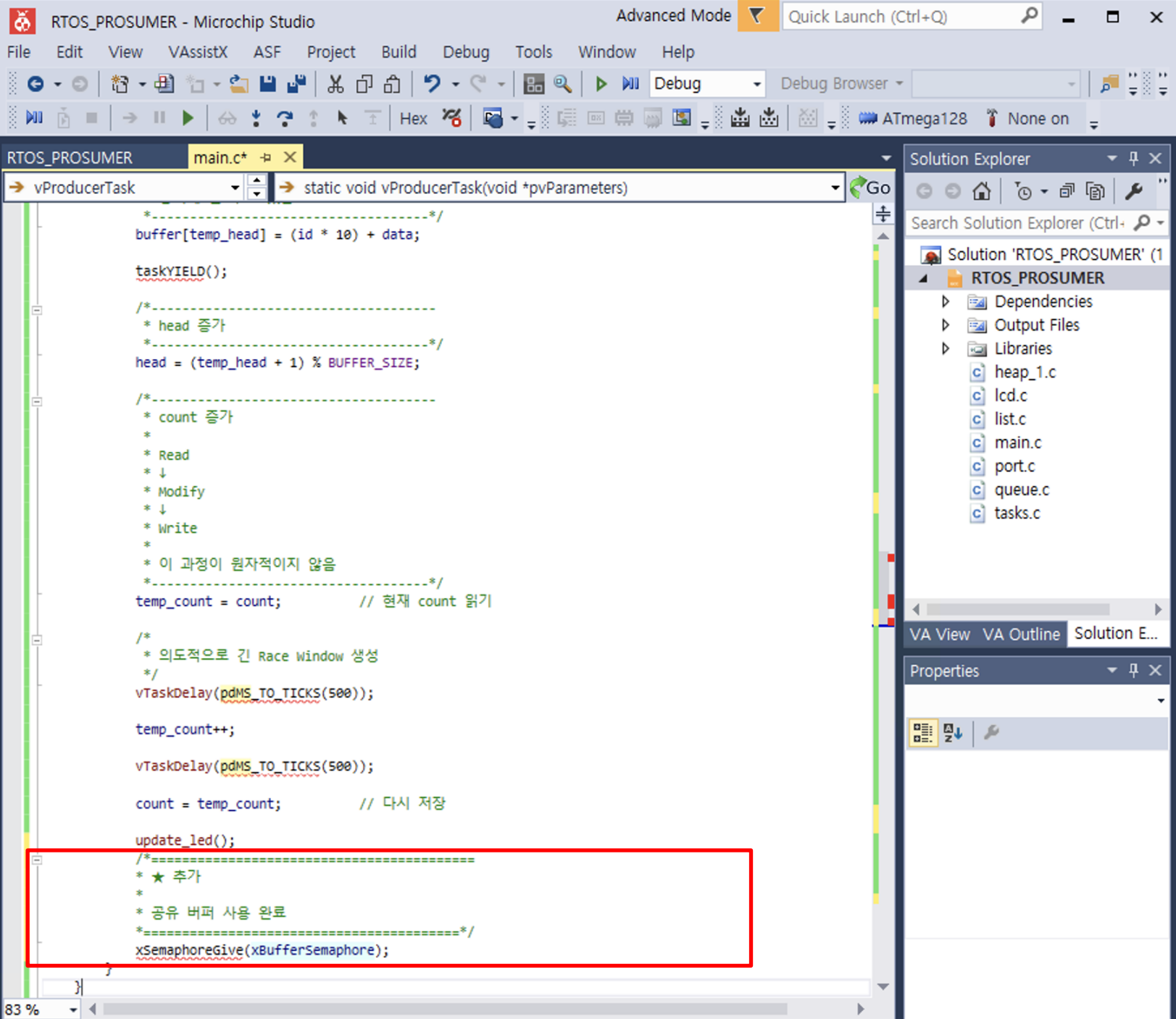
Solution 'RTOS_PROSUMER' (1)

- RTOS_PROSUMER
 - Dependencies
 - Output Files
 - Libraries
 - heap_1.c
 - lcd.c
 - list.c
 - main.c
 - port.c
 - queue.c
 - tasks.c

VA View VA Outline Solution E...

Properties





생산자 소비자 문제

☐ main.c

○ vConsumerTask 함수도 수정!!!!

RTOS_PROSUMER - Microchip Studio

Advanced Mode Quick Launch (Ctrl+Q)

File Edit View VAssistX ASF Project Build Debug Tools Window Help

Debug Debug Browser

ATmega128 None on

RTOS_PROSUMER main.c*

random_delay static uint16_t random_delay(uint16_t *seed, uint16_t min, uint16_t max) Go

```
/*
 * 랜덤 대기 시간
 * min ~ max ms
 */
static uint16_t random_delay(uint16_t *seed, uint16_t min, uint16_t max) {
    uint16_t range;

    range = max - min + 1;

    return min + (random_number(seed) % range);
}

static void vLCDTask(void *pvParameters) {
    uint8_t n;

    (void)pvParameters;

    while (1) {
        /*
         * 현재 count 값을 복사
         */
        xSemaphoreTake(xBufferSemaphore, portMAX_DELAY);

        n = count;

        /* ★ 추가 */
        xSemaphoreGive(xBufferSemaphore);

        /*
         * 두 번째 줄에 출력
         */
        lcd_gotoxy(0, 1);
        lcd_string("Count = ");

        /*
         * 0~9라면 한 자리 출력
         */
        if (n <= 9) { lcd_data('0' + n); }
        else { lcd_data(' '); }

        /*
         * 이전 문자 제거용 공백
         */
    }
}
```

83 %

Solution Explorer

Search Solution Explorer (Ctrl+)

Solution 'RTOS_PROSUMER' (1)

RTOS_PROSUMER

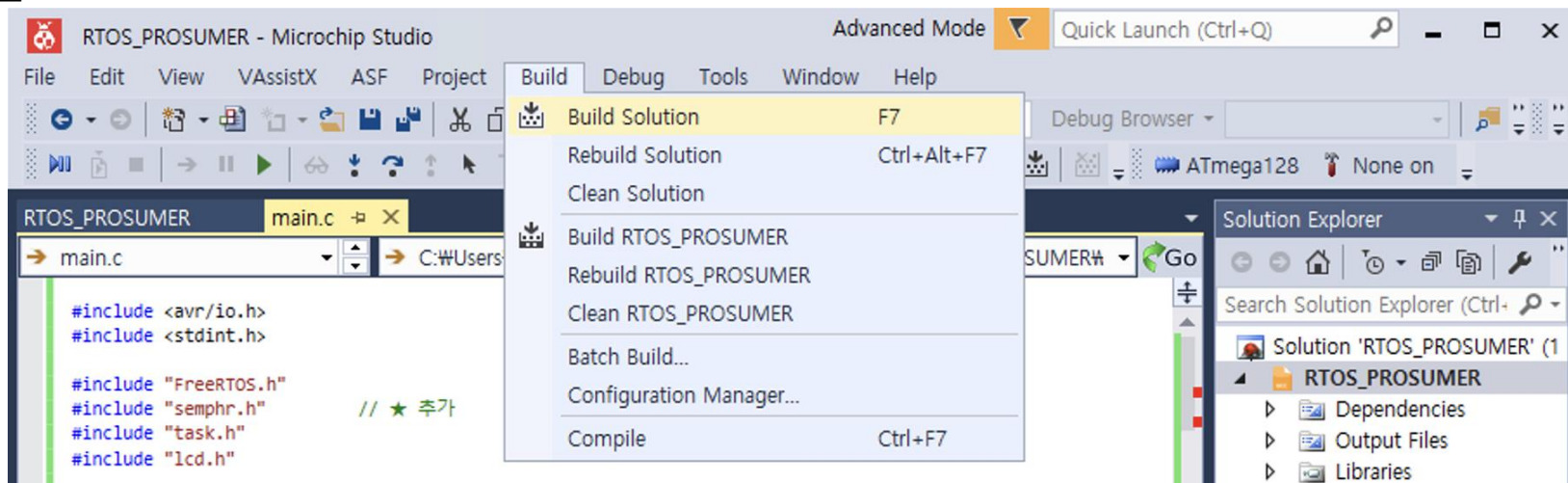
- Dependencies
- Output Files
- Libraries
- heap_1.c
- lcd.c
- list.c
- main.c
- port.c
- queue.c
- tasks.c

VA View VA Outline Solution E...

Properties

생산자 소비자 문제

□ Build



□ Device Programming

